

EX-ACT Django Webapp - Developer Setup Guide

This guide walks you through setting up the EX-ACT Django web application from scratch on your local machine. It covers everything from installing Python and VS Code to debugging emission calculations.

Last verified: 2026-05 on macOS 14 (Apple Silicon) and Windows 11. Cloud SQL Proxy version pinned at **v2.14.3** (newer versions in the **v2.x** line should work too).

What success looks like

By the end of this guide you will have:

1. The Django dev server running at **http://localhost:8000/**.
2. A working login at **http://localhost:8000/admin/** with a superuser you created.
3. A VS Code debugger that pauses on a breakpoint inside **djangoexact/math_model/no_time_dependency_final/annuals.py** when you make an annual-cropland API request.

If any of these three is missing, you are not done - jump to [Troubleshooting](#).

Repository layout (memorise this)

```

exact-django-webapp/      ← repo root. venv lives here. .vscode/launch.json
lives here.
├─ djangoexact/          ← Django root. manage.py + requirements.txt live
here.
│   └─ djangoexact/      ← Django settings package. .env files live here.
│       ├── settings.py
│       └── .env / .env.development / .env.review / .env.production
│   └─ api/calculators.py ← Calculator layer (one class per Django model)
│   └─ math_model/no_time_dependency_final/ ← Pure math, the actual emission
logic
│       ├── package.json ← Frontend assets (Webpack + Tailwind + React)
│       └── manage.py
├─ docs/                 ← This guide lives here.
└─ .vscode/launch.json   ← Debugger config (you will create this)
  
```

You will be **cd**-ing between these two levels a lot. **Every code block in this guide is annotated with the directory it must be run from** - read those comments.

Shell conventions

Commands are written in three flavours depending on your OS:

- **macOS / Linux** - bash or zsh.
- **Windows (PowerShell)** - the default on Windows 11. Use **\$env:VAR="value"** to set environment variables.
- **Windows (cmd.exe)** - only shown when PowerShell syntax differs.

Pick one shell and stick with it for the whole session. Mixing them is the #1 cause of mysterious `APP_MODE` failures.

Who to ask for what

What you need	Who has it
Populated <code>.env.*</code> files (DB credentials, Firebase keys)	Any team member already running locally
GCP project access (dev / review / production)	Project lead / FAO CSI Information Office
Firebase project access	Project lead
Superuser credentials for shared environments	Project lead - never reuse production credentials locally

Table of Contents

1. Prerequisites
 - [Install Python 3.11](#)
 - [Install VS Code](#)
 - [Install VS Code Extensions](#)
 - [Install Google Cloud SQL Proxy](#)
 - [Install Node.js & npm](#)
 - [Install Git](#)
 - [WeasyPrint system libraries](#)
2. Project Setup
 - [Clone the Repository](#)
 - [Create a Virtual Environment](#)
 - [Install Python Dependencies](#)
 - [Point VS Code at the venv](#)
 - [Build Frontend Assets](#)
 - [Initialise the Database](#)
3. Environment Configuration
 - [How Environment Files Work](#)
 - [Getting Your .env Files](#)
 - [Switching Between Environments](#)
 - [Verifying Your .env Loaded](#)
4. Database Access with Cloud SQL Proxy
 - [Authenticating with GCloud](#)
 - [Available Database Environments](#)
 - [Running the Proxy](#)
 - [Making Production Actually Read-Only](#)
5. Running the Application
 - [Start the Server](#)
 - [Smoke Test](#)
 - [Authenticated API Requests](#)

- 6. [Debugging in VS Code](#)
 - [Setting Up the Debugger](#)
 - [Using Breakpoints](#)
 - [Debugging the Calculators & Math Model](#)
 - [Using the Debug Variables Panel](#)
 - [Practical Debugging Walkthrough](#)
 - 7. [Troubleshooting](#)
 - 8. [Day-2 Operations](#)
 - [Tests](#)
 - [Branching & Pull Requests](#)
 - [Fixtures & Reference Data](#)
-

1. Prerequisites

1.1 Install Python 3.11

This project requires **Python 3.11.x**. Other versions may cause compatibility issues with dependencies.

macOS (using Homebrew):

```
brew install python@3.11
```

Verify the installation:

```
python3.11 --version
```

You should see **Python 3.11.x**.

Windows:

1. Download the Python 3.11.x installer from python.org
2. Run the installer - **check "Add python.exe to PATH"** before clicking Install
3. Open a new terminal and verify:

```
python --version
```

1.2 Install VS Code

1. Download VS Code from code.visualstudio.com
2. Run the installer for your platform
3. Launch VS Code

1.3 Install VS Code Extensions

Open VS Code, go to the Extensions sidebar (**Cmd+Shift+X** on macOS, **Ctrl+Shift+X** on Windows), and install the following:

Extension	Publisher	Purpose
Python	Microsoft	Python language support, IntelliSense, linting
Pylance	Microsoft	Fast, feature-rich Python language server
Django	Baptiste Darthenay	Django template syntax highlighting
Python Debugger	Microsoft	Breakpoints, variable inspection, step-through
GitLens	GitKraken	Git blame, history, and navigation
Thunder Client	Ranga Vadhineni	REST API testing (alternative to Postman)
SQLTools	Matheus Teixeira	Database browser and query runner
SQLTools PostgreSQL	Matheus Teixeira	PostgreSQL driver for SQLTools

You can also install them from the terminal:

```
code --install-extension ms-python.python
code --install-extension ms-python.vscode-pylance
code --install-extension batisteo.vscode-django
code --install-extension ms-python.debugpy
code --install-extension eamodio.gitlens
code --install-extension rangav.vscode-thunder-client
code --install-extension mtxr.sqltools
code --install-extension mtxr.sqltools-driver-pg
```

1.4 Install Google Cloud SQL Proxy

The Cloud SQL Proxy lets you connect to remote GCP PostgreSQL databases as if they were running locally.

macOS (Apple Silicon):

```
curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.14.3/cloud-sql-proxy.darwin.arm64
chmod +x cloud-sql-proxy
```

macOS (Intel):

```
curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.14.3/cloud-sql-proxy.darwin.amd64
chmod +x cloud-sql-proxy
```

Linux (x64):

```
curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.14.3/cloud-sql-proxy.linux.amd64
chmod +x cloud-sql-proxy
```

Windows (PowerShell):

```
# From any directory; we'll move the file into a permanent home below.
Invoke-WebRequest `
  -Uri https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.14.3/cloud-sql-proxy.x64.exe `
  -OutFile cloud-sql-proxy.exe
```

Move the binary somewhere stable and add it to PATH.

macOS / Linux:

```
mv cloud-sql-proxy ~/Developer/cloud-sql-proxy
# or, system-wide:
sudo mv cloud-sql-proxy /usr/local/bin/cloud-sql-proxy
```

Windows (PowerShell, run as Administrator if installing system-wide):

```
# Create a dedicated tools folder under your user profile
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\Developer" | Out-Null
Move-Item cloud-sql-proxy.exe "$env:USERPROFILE\Developer\cloud-sql-proxy.exe"

# Add that folder to PATH for the current user (persists across sessions)
[Environment]::SetEnvironmentVariable(
  "Path",
  [Environment]::GetEnvironmentVariable("Path", "User") +
  ";$env:USERPROFILE\Developer",
  "User"
)
# Close and reopen the terminal for the new PATH to take effect.
```

Verify the install:

```
cloud-sql-proxy --version    # any OS, after PATH is set
```

Windows note: every command in this guide that shows `~/Developer/cloud-sql-proxy` should be read as `cloud-sql-proxy.exe` (or just `cloud-sql-proxy` once it's on PATH).

Install the Google Cloud CLI if you don't have it:

macOS:

```
brew install --cask google-cloud-sdk
```

Linux:

```
curl https://sdk.cloud.google.com | bash  
exec -l $SHELL
```

Windows: download and run [GoogleCloudSDKInstaller.exe](#). After installation, **close and reopen your terminal** before running `gcloud` for the first time - the installer modifies PATH but existing shells won't pick that up.

Verify:

```
gcloud --version
```

1.5 Install Node.js & npm

The project ships a small frontend bundle (Webpack + Tailwind + a few React components). You won't write much JS in day-to-day work, but you need to **build the assets at least once** or the admin and a few pages will render unstyled.

- **macOS:** `brew install node@20`
- **Windows:** [nodejs.org](#) installer (LTS).
- **Linux:** use `nvm` and install Node 20 LTS.

Verify:

```
node --version    # v20.x or newer  
npm --version
```

1.6 Install Git

- **macOS:** comes with Xcode Command Line Tools - `xcode-select --install`.
- **Windows:** [Git for Windows](#). During install, accept the default "Git Bash" - useful as a fallback shell.
- **Linux:** `sudo apt install git` / `sudo dnf install git`.

Verify:

```
git --version
```

1.7 WeasyPrint system libraries

WeasyPrint is used to render PDF emission reports. Its Python wheel needs system libraries that pip cannot install:

macOS:

```
brew install pango gdk-pixbuf libffi
```

Linux (Debian/Ubuntu):

```
sudo apt install -y libpango-1.0-0 libpangoft2-1.0-0 libffi-dev
```

Windows: install the [GTK 3 runtime](#) before running `pip install`. If that fails, the path of least resistance is to develop inside [WSL2](#) - the Linux instructions then apply.

You won't hit WeasyPrint errors until you exercise the report endpoints, but installing the libraries now avoids a painful detour later.

2. Project Setup

2.1 Clone the Repository

```
# from: wherever you keep code, e.g. ~/Developer/  
git clone <repository-url>  
cd exact-django-webapp
```

All subsequent commands assume you are inside `exact-django-webapp/` unless the comment above the command says otherwise.

2.2 Create a Virtual Environment

Important: the venv is created at the **repo root** (`exact-django-webapp/`), but `manage.py` and `requirements.txt` live one level deeper in `djangoexact/`. You will be activating the venv at the root and `cd`-ing into `djangoexact/` to run Django commands. This trips up almost every new hire - read it twice.

```
# from: exact-django-webapp/  
python3.11 -m venv venv
```

On Windows the `python3.11` alias may not exist; use `py -3.11 -m venv venv` instead.

Activate it:

macOS / Linux (bash/zsh):

```
source venv/bin/activate
```

Windows (PowerShell):

```
.\venv\Scripts\Activate.ps1
```

If PowerShell refuses with an execution-policy error, run once in an Admin PowerShell:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Windows (cmd.exe):

```
venv\Scripts\activate.bat
```

You should see `(venv)` in your terminal prompt.

2.3 Install Python Dependencies

```
# from: exact-django-webapp/djangoexact/  
cd djangoexact  
pip install -r requirements.txt
```

This installs Django 4.2, DRF, numpy, pandas, WeasyPrint, and all other project dependencies. The installation may take several minutes the first time.

If `psycpg2-binary` fails to install:

- **macOS:** `brew install libpq && pip install psycpg2-binary --no-cache-dir`
- **Linux:** `sudo apt install libpq-dev && pip install psycpg2-binary --no-cache-dir`
- **Windows:** make sure you are using the **64-bit** Python 3.11; the precompiled wheel only exists for x64. If it still fails, install [Microsoft C++ Build Tools](#) and retry.

2.4 Point VS Code at the venv

If you skip this step, VS Code will run scripts and the debugger against your **system** Python, none of the project dependencies will be importable, and breakpoints will silently fail to bind. This is by far the most common day-one problem.

1. Open the `exact-django-webapp/` folder in VS Code.
2. `Ctrl+Shift+P` (`Cmd+Shift+P` on macOS) → **Python: Select Interpreter**.

- Pick the entry that ends in `venv\Scripts\python.exe` (Windows) or `venv/bin/python` (macOS / Linux).
- Open any `.py` file in `djangoexact/` and confirm the bottom-right status bar shows `Python 3.11.x ('venv')`.

Persist the choice for the workspace by creating `.vscode/settings.json` at the repo root with:

```
{
  "python.defaultInterpreterPath": "${workspaceFolder}/venv/bin/python",
  "python.envFile": "${workspaceFolder}/djangoexact/djangoexact/.env.development",
  "python.terminal.activateEnvironment": true
}
```

On Windows, change the interpreter path to `${workspaceFolder}/venv/Scripts/python.exe`.

2.5 Build Frontend Assets

The Django templates depend on a bundled JS/CSS file produced by Webpack. Build it once now:

```
# from: exact-django-webapp/djangoexact/
npm install
npm run build
```

`npm run build` produces a development bundle. The only npm script defined today is `build` (see `djangoexact/package.json`); re-run it whenever you touch frontend code, or wrap it in a watcher of your choice.

2.6 Initialise the Database

Once your `.env.development` is in place (next section) **and** the Cloud SQL Proxy is running (Section 4), apply migrations and create a superuser:

```
# from: exact-django-webapp/djangoexact/
APP_MODE=development python manage.py migrate
APP_MODE=development python manage.py createsuperuser
```

(For the PowerShell/cmd equivalents of the `APP_MODE=...` prefix, see §3.3.)

Load reference data so the IPCC default-value tables and the country / region lookups exist. Without this, half the calculators will raise on null lookups and you will think the math model is broken:

```
APP_MODE=development python manage.py load_reference_data --app=all
```

See `djangoexact/docs/guides/fixtures-guide.md` for the full dump/load workflow.

3. Environment Configuration

3.1 How Environment Files Work

The application uses `python-dotenv` with a multi-layered environment loading strategy defined in `djangoexact/djangoexact/settings.py`:

1. A base `.env` file is loaded first (general defaults)
2. If the `APP_MODE` environment variable is set, it then loads `.env.{APP_MODE}` which overrides the base values

The environment files live at:

<code>djangoexact/djangoexact/.env</code>	← base (always loaded)
<code>djangoexact/djangoexact/.env.development</code>	← local development
<code>djangoexact/djangoexact/.env.review</code>	← review environment
<code>djangoexact/djangoexact/.env.production</code>	← production environment
<code>djangoexact/djangoexact/.env.test</code>	← test environment

These files are **gitignored** and contain secrets (database credentials, Firebase keys, etc.), so they are never committed to the repository.

Recommended split: keep environment-invariant values (`EMAIL_*`, `FIREBASE_*`, `STORAGE_BUCKET`) in the base `.env`, and put per-environment values (`DB_*`, `ALLOWED_HOSTS`, `CORS_ALLOWED_ORIGINS`) in each `.env.{mode}`. The override semantics mean any value set in both files takes its value from the mode-specific file.

3.2 Getting Your .env Files

The repo ships a fully-annotated template at `djangoexact/.env.example`. Use it as a reference for which variables exist and what they mean.

Option A - fastest (recommended): ask a team member for the pre-populated `.env`, `.env.development`, `.env.review`, and `.env.production` files. Drop them into `djangoexact/djangoexact/`. Done.

Option B - from scratch: copy `djangoexact/.env.example` into `djangoexact/djangoexact/.env.development` and fill in the placeholders.

```
# from: exact-django-webapp/  
cp djangoexact/.env.example djangoexact/djangoexact/.env.development
```

The two values you cannot guess at are `FIREBASE_SERVICE_ACCOUNT` and the database credentials - see the help comments in `.env.example`.

Obtaining `FIREBASE_SERVICE_ACCOUNT`:

1. Go to the Firebase console → ⚙ **Project settings** → **Service accounts**.

2. Click **Generate new private key** → save the downloaded JSON.

3. Base64-encode the file:

macOS / Linux:

```
base64 -i path/to/service-account.json
```

Windows (PowerShell):

```
[Convert]::ToBase64String([IO.File]::ReadAllBytes("path\to\service-account.json"))
```

4. Paste the single-line output as the value of `FIREBASE_SERVICE_ACCOUNT=...` in your `.env`.

3.3 Switching Between Environments

Set the `APP_MODE` environment variable before running the server. The exact syntax depends on your shell:

Shell	One-shot command	Export for the whole session
bash / zsh	<code>APP_MODE=development python manage.py runserver</code>	<code>export APP_MODE=development</code>
PowerShell	<code>\$env:APP_MODE="development"; python manage.py runserver</code>	<code>\$env:APP_MODE="development"</code>
cmd.exe	<code>set APP_MODE=development && python manage.py runserver</code>	<code>set APP_MODE=development</code>

The three database modes are:

- `APP_MODE=development` - **dev** GCP project (`fao-exact-dev`).
- `APP_MODE=review` - **review** GCP project (`fao-exact-review`). Use before merging to production.
- `APP_MODE=production` - **live** data. Read-only by convention; see §4.4 for how to actually enforce it.

If you forget to set `APP_MODE`, only the base `.env` is loaded. The server may start but will probably fail on the first request because no DB connection details were filled in. Watch the first line of stdout - Django prints `Running in {mode} mode` when `APP_MODE` is set.

3.4 Verifying Your .env Loaded

Quick sanity check that the right file was picked up:

```
# from: exact-django-webapp/djangoexact/
APP_MODE=development python manage.py shell -c "from django.conf import settings;
```

```
print(settings.DATABASES['default']['HOST'], settings.DATABASES['default']
['NAME'])"
```

If you see `127.0.0.1 <the-dev-database-name>` you're good. If you see literal placeholders like `$DB_HOST`, the `.env` is not being read - most often because the file is at the wrong path (must be `djangoexact/djangoexact/.env.development`, **not** `djangoexact/.env.development`).

4. Database Access with Cloud SQL Proxy

The EX-ACT databases are hosted on Google Cloud SQL (PostgreSQL). To access them locally, you need to run the Cloud SQL Proxy, which creates a secure tunnel.

4.1 Authenticating with GCloud

Before using the proxy, authenticate with Google Cloud:

```
gcloud auth login                # signs you in for gcloud CLI commands
gcloud auth application-default login  # writes credentials the proxy will
read
```

This opens a browser window for authentication. Make sure you log in with your FAO Google account that has access to the GCP projects.

The second command writes an **Application Default Credentials** file at:

- macOS / Linux: `~/.config/gcloud/application_default_credentials.json`
- Windows: `%APPDATA%\gcloud\application_default_credentials.json`

The Cloud SQL Proxy reads that file at startup. If you ever see `could not find default credentials` from the proxy, this file is missing or stale - rerun the second command.

4.2 Available Database Environments

Environment	GCP Project	Instance Connection	Use Case
Dev	<code>fao-exact-dev</code>	<code>fao-exact-dev:eu-west1:fao-exact-dev-postgres</code>	Day-to-day development
Review	<code>fao-exact-review</code>	<code>fao-exact-review:eu-west1:fao-exact-review-postgres</code>	Testing before production
Production	<code>fao-exact</code>	<code>fao-exact:eu-west1:fao-exact-postgres</code>	Live data (be careful!)

4.3 Running the Proxy

The proxy binds to `localhost:5432`, making the remote database appear as a local PostgreSQL instance.

Dev environment:

```
~/Developer/cloud-sql-proxy -p 5432 fao-exact-dev:europa-west1:fao-exact-dev-postgres
```

Review environment:

```
~/Developer/cloud-sql-proxy fao-exact-review:europa-west1:fao-exact-review-postgres --port=5432
```

Production environment:

```
~/Developer/cloud-sql-proxy -p 5432 fao-exact:europa-west1:fao-exact-postgres
```

Keep the proxy running in a separate terminal tab/window. It must stay active while you work with the database.

Once the proxy is running, your `.env` file's `DB_HOST=127.0.0.1` and `DB_PORT=5432` will route through the proxy to the remote database.

Windows tip: to keep the proxy out of your way, run it in a [Windows Terminal](#) split pane (Alt+Shift++), or wrap the command in `Start-Process -WindowStyle Hidden` if you don't need to read its output. Either way, **the proxy must stay running** the whole time you work with the database.

To verify the connection works:

```
# from: exact-django-webapp/djangoexact/  
APP_MODE=development python manage.py dbshell
```

This should drop you into a PostgreSQL shell connected to the remote database. Type `\q` to exit.

4.4 Making Production Actually Read-Only

The "Production (READ-ONLY)" label in `launch.json` is a convention, not an enforcement. To make it physically impossible to write to production while connected through your local machine, do one of:

1. **Use a read-only Postgres role.** Ask the project lead for an `exact_readonly` user and put its credentials in `.env.production`. This is the recommended approach.
2. **Force every transaction to be read-only** in `.env.production` by setting `DB_OPTIONS_OPTIONS='-c default_transaction_read_only=on'` (the project's `settings.py` honours `DB_OPTIONS_*` env vars if you wire them in; if it doesn't yet, file a beads issue rather than skipping this step).
3. **Don't run with `APP_MODE=production` at all** unless you have a documented reason. Use `review` for almost everything.

Any **UPDATE**, **INSERT**, or **DELETE** you accidentally run with full credentials will land on live user data with no undo. Treat the production proxy connection as a footgun.

5. Running the Application

5.1 Start the Server

With the proxy running and your **.env** file configured:

```
# from: exact-django-webapp/djangoexact/  
APP_MODE=development python manage.py runserver
```

The server starts at <http://localhost:8000/>. Key URLs:

URL	Description
http://localhost:8000/admin/	Django Admin panel
http://localhost:8000/api/	API root
http://localhost:8000/api/docs/	API documentation
http://localhost:8000/api/swagger/	Swagger UI
http://localhost:8000/api/redoc/	ReDoc documentation

If port 8000 is already in use (common on Windows when Hyper-V or IIS grabs it), pick another:

```
APP_MODE=development python manage.py runserver 0.0.0.0:8001
```

5.2 Smoke Test

Don't move on until **all four** of these pass:

Check	Pass criterion
GET http://localhost:8000/admin/	Returns the Django admin login page (HTTP 200, not 500).
Log in with the superuser from §2.6	Lands on the admin dashboard.
GET http://localhost:8000/api/docs/	Renders Swagger UI with a non-empty endpoint list.
Terminal where runserver is running	Logged Running in development mode at startup; no red tracebacks during the requests above.

If any of these fails, jump to [Troubleshooting](#) before continuing.

5.3 Authenticated API Requests

Most API endpoints require token authentication. To grab a token for your superuser:

```
curl -X POST http://localhost:8000/api/auth/token/ \
-H "Content-Type: application/json" \
-d '{"username": "<your-superuser>", "password": "<your-password>"}'
```

Use the returned token as **Authorization: Token <token>** in Thunder Client / Postman / `curl`. Save the token in a Thunder Client environment variable so you don't have to paste it for every request.

6. Debugging in VS Code

6.1 Setting Up the Debugger

Create the file `.vscode/launch.json` in the **project root** with this configuration:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Django: Dev",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/djangoexact/manage.py",
      "args": ["runserver", "--noreload"],
      "django": true,
      "env": {
        "APP_MODE": "development"
      },
      "justMyCode": false
    },
    {
      "name": "Django: Review",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/djangoexact/manage.py",
      "args": ["runserver", "--noreload"],
      "django": true,
      "env": {
        "APP_MODE": "review"
      },
      "justMyCode": false
    },
    {
      "name": "Django: Production (READ-ONLY)",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/djangoexact/manage.py",
      "args": ["runserver", "--noreload"],
      "django": true,
    }
  ]
}
```

```

    "env": {
        "APP_MODE": "production"
    },
    "justMyCode": false
}
]
}

```

Key settings explained:

- **--noreload**: Prevents Django's auto-reloader from interfering with the debugger. Without this, breakpoints may not work reliably.
- **"django": true**: Enables Django-specific debugging support (template debugging, etc.).
- **"justMyCode": false**: Allows stepping into library code and the math model modules, which is essential for debugging calculations. **Leave this as false** - if you flip it to **true**, the debugger will silently skip over `math_model/` files and your breakpoints will appear "unverified".
- **"env"**: Sets the `APP_MODE` so the correct `.env.{mode}` file is loaded.

6.2 Using Breakpoints

1. Open any Python file in VS Code
2. Click in the gutter (left of the line numbers) to place a red dot - that's a breakpoint
3. Start the debugger with **F5** (or click the green play button in the Run & Debug sidebar)
4. Trigger the code path (e.g. make an API request) - execution pauses at your breakpoint

Breakpoint types:

Type	How to Set	Use Case
Line breakpoint	Click the gutter	Pause at a specific line
Conditional breakpoint	Right-click gutter → "Add Conditional Breakpoint"	Pause only when a condition is true (e.g. <code>module_name == "AnnualCropland"</code>)
Logpoint	Right-click gutter → "Add Logpoint"	Log a message without pausing execution
Exception breakpoint	In the Breakpoints panel, check "Raised Exceptions"	Pause when any exception is thrown

6.3 Debugging the Calculators & Math Model

The EX-ACT calculation pipeline follows this flow:

```

API Request
→ View / ViewSet
→ Calculator (djangoexact/api/calculators.py)
  → Math Model (djangoexact/math_model/no_time_dependency_final/*.py)
    → Result / Inventory objects

```

The Calculators Layer (`djangoexact/api/calculators.py`)

Each Django model has a corresponding calculator class (e.g. `AnnualCroplandCalculator`, `DeforestationCalculator`, `ForestManagementCalculator`). These calculators:

1. Fetch input data from Django model instances
2. Look up IPCC default values
3. Build keyword-argument dictionaries (`self.inputs_w`, `self.inputs_wo`)
4. Instantiate the math model class (e.g. `MathAnnualCropland(**self.inputs_w)`)
5. Call `calculate_emissions()` on the math model
6. Return the `MathResult` object

Good breakpoint locations in `calculators.py`:

- Inside any calculator's `calculate()` method - right before the math model is instantiated
- On the line where `calculate_emissions()` is called
- On the `return` statement to inspect final results

The Math Model Layer (`djangoexact/math_model/no_time_dependency_final/`)

The math model files contain the actual emission calculation logic:

File	Class	What It Calculates
<code>annuals.py</code>	<code>AnnualCropland</code>	Annual cropland emissions (SOC, biomass, N ₂ O)
<code>defo.py</code>	<code>Defo</code>	Deforestation emissions
<code>forest_management.py</code>	<code>ForestManagement</code>	Forest management (AGB/BGB matrices)
<code>perennial_cropping.py</code>	<code>PerennialTreeCrop</code>	Perennial/tree crop emissions
<code>flooded_rice.py</code>	<code>FloodedRice</code>	Flooded rice methane emissions
<code>grassland_management.py</code>	<code>GrasslandManagement</code>	Grassland SOC and biomass changes
<code>coastal_wetlands.py</code>	<code>CoastalWetland</code>	Mangrove and tidal marsh emissions
<code>fisheries_and_aquaculture.py</code>	<code>Fishery</code> , <code>CoastalAquaculture</code>	Fisheries fuel and aquaculture emissions
<code>inlands.py</code>	<code>AnnexedModule</code> , <code>PeatExtraction</code>	Organic soils, peat drainage/rewetting

File	Class	What It Calculates
<code>inputs.py</code>	<code>ElectricityConsumption</code> , <code>SolidAndLiquidFuelsConsumption</code> , etc.	Energy and input emissions
<code>livestock.py</code>	<code>Livestock</code>	Enteric fermentation, manure management
<code>value_chains.py</code>	Various	Storage, processing, packaging, transport
<code>waterbodies.py</code>	<code>Waterbodies</code>	Reservoir and waterbody emissions

All math model classes inherit from `BaseModule` or `LandModule` (defined in `generalized_modules.py`) and share the same pattern:

- `__post_init__()` initializes `self.result` (a `MathResult`) and `self.inventory` (an `Inventory`)
- `calculate_emissions()` populates them

The shape of `self.result` (what you'll see in the Variables panel):

```

MathResult
├─ yearly_emissions_by_sector_by_gas : list[YearlyGasActivityEmissionSet]
│   └─ .year : int - 0-based year index inside the project
horizon
├─ .gas_type : "CO2" | "CH4" | "N2O"
├─ .activity : str - e.g. "biomass_burning", "soil_emissions"
├─ .emissions : list[Emission]
│   └─ .value : float - ← this is the number you're chasing
└─ (module-specific fields, e.g. soc_start, soc_end, hectares_total)

Inventory - available as self.inventory
├─ emissions_by_sector_by_gas : list[InventoryPerGasPerActivity]
│   └─ .gas_type / .activity
│       └─ .value : float - aggregated total across all years

```

Knowing this shape up-front means you can navigate the Variables panel without expanding every node.

Good breakpoint locations in math model files:

- Inside `calculate_emissions()` methods - step through the calculation logic
- Inside `general_functions.py` helper functions like `compute_yearly_or_half_year_cumulative()`, `soil_emissions()`, `biomass_emissions()`

6.4 Using the Debug Variables Panel

When execution is paused at a breakpoint, the **Variables** panel (left sidebar in the Debug view) is your most powerful tool.

Locals

Shows all variables in the current scope. For a calculator method, you'll see:

- `self` - the calculator instance
- `self.inputs_w` / `self.inputs_wo` - the dictionaries of inputs passed to the math model (inspect these to verify the right values are being sent)

Inspecting `self` on Math Model Objects

When paused inside a math model's `calculate_emissions()`:

- `self.result` - the `MathResult` object being built
 - `self.result.yearly_emissions_by_sector_by_gas` - list of `YearlyGasActivityEmissionSet` objects, each with:
 - `.year` - the year index
 - `.gas_type` - the greenhouse gas (`CO2`, `CH4`, `N2O`)
 - `.activity` - the activity type
 - `.emissions` - list of `Emission` objects with `.value`
- `self.inventory` - the `Inventory` object
 - `self.inventory.emissions_by_sector_by_gas` - list of `InventoryPerGasPerActivity` objects with `.gas_type`, `.value`, `.activity`

Watch Expressions

Add custom watch expressions in the **Watch** panel (click the `+` button):

Useful watches when debugging calculations:

```
self.result.yearly_emissions_by_sector_by_gas
len(self.result.yearly_emissions_by_sector_by_gas)
self.hectares_total
self.soc_start
self.soc_end
self.implementation_time
self.capitalization_time
```

Debug Console

The **Debug Console** (`Cmd+Shift+Y` / `Ctrl+Shift+Y`) lets you evaluate arbitrary Python expressions while paused:

```
# Check the total emissions across all years for a specific gas
sum(e.value for yset in self.result.yearly_emissions_by_sector_by_gas
    for e in yset.emissions)

# Inspect a specific input dictionary
self.inputs_w.keys()
```

```
# Check what IPCC values were fetched
self.inputs_w.get('soc_reference')
```

6.5 Practical Debugging Walkthrough

Here's a step-by-step example of debugging an annual cropland calculation:

1. Set breakpoints:

- `calculators.py` - inside `AnnualCroplandCalculator.calculate()`, on the line that creates `MathAnnualCropland(**self.inputs_w)`
- `annuals.py` - inside `AnnualCropland.calculate_emissions()`, at the first line

2. Start the debugger (F5, select "Django: Dev")

3. Trigger the calculation by making an API request (via browser, Thunder Client, or Postman) that invokes the annual cropland calculation

4. Execution pauses at the first breakpoint in `calculators.py`:

- In the **Variables** panel, expand `self.inputs_w` to see all the keyword arguments
- Verify values like `hectares`, `implementation_years`, `soc_reference`, `flu`, `fmg`, `fi` are correct
- Check if any Tier 2 overrides are set (`*_tier_2` fields)

5. Press F5 (Continue) - execution moves to the breakpoint in `annuals.py`

6. Step through with F10 (Step Over) or F11 (Step Into):

- Watch `self.soc_start` and `self.soc_end` get calculated
- Step into `self.calculate_soc_som()` to see soil organic carbon computation
- Step into `general_functions.soil_emissions()` to see the yearly emission curves

7. Check the result:

- After `calculate_emissions()` completes, inspect `self.result` in the Variables panel
- Expand `yearly_emissions_by_sector_by_gas` to see per-year, per-gas emissions
- Check `self.inventory.emissions_by_sector_by_gas` for the aggregated totals

Debug Keyboard Shortcuts

Shortcut	Action
F5	Start / Continue
F10	Step Over (next line, skip function internals)
F11	Step Into (go inside a function call)
Shift+F11	Step Out (finish current function, return to caller)
Shift+F5	Stop debugging
Cmd+Shift+F5	Restart debugging

7. Troubleshooting

"Firebase config not found" / `binascii.Error: Invalid base64-encoded string` on startup

The Firebase service account is required even for local development. Make sure `FIREBASE_SERVICE_ACCOUNT` is set in your `.env` file and is valid base64.

```
# Test your base64 value (macOS / Linux)
echo "your-base64-value" | base64 -d | python -m json.tool
```

```
# Test your base64 value (PowerShell)
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String("your-base64-value"))
| ConvertFrom-Json
```

A common mistake on Windows is that `base64` in some shells inserts line breaks every 76 chars - the value must be a **single line** in the `.env`.

Cloud SQL Proxy: "connection refused"

- Make sure the proxy is **running** in a separate terminal (look for `Ready for new connections`).
- Verify you're authenticated: `gcloud auth application-default login`.
- Check that no other process is using port 5432:
 - macOS / Linux: `lsof -i :5432`
 - Windows: `Get-NetTCPConnection -LocalPort 5432`
- If PostgreSQL is installed locally and listening on 5432, stop it (`brew services stop postgresql` / `Services.msc`) or run the proxy on a different port (`--port=5433`, then update `DB_PORT` in your `.env`).

`gcloud: command not found` on Windows after install

The installer modifies PATH but already-open terminals don't see it. **Close and reopen** the terminal, or restart your VS Code window. If it still doesn't appear, run `gcloud` once via its full path to confirm the install:

```
& "$env:LOCALAPPDATA\Google\Cloud SDK\google-cloud-sdk\bin\gcloud.cmd" --version
```

`pip install psycopg2-binary` fails on Windows

You almost certainly have 32-bit Python. Reinstall the 64-bit Python 3.11 from python.org. If you genuinely need 32-bit, install Microsoft C++ Build Tools and retry; there is no precompiled 32-bit wheel for recent `psycopg2` versions.

"No module named 'math_model'"

You need to run from the `djangoexact/` directory, where `math_model/` is a direct subdirectory:

```
# from: exact-django-webapp/djangoexact/
python manage.py runserver
```

load_reference_data fails on a fresh DB

`migrate` must complete successfully first. If it has, rerun with verbose output to see which fixture is choking:

```
APP_MODE=development python manage.py load_reference_data --app=all --verbosity=2
```

Breakpoints not hitting / shown as "unverified"

In rough order of likelihood:

1. **Wrong interpreter in VS Code.** Check the bottom-right status bar - it must show ('venv'). If it doesn't, run §2.4 again.
2. **Auto-reloader interfering.** Make sure `--noreload` is in your launch arg list.
3. **justMyCode set to true.** Breakpoints inside `math_model/`, `general_functions.py`, or any installed package will be skipped silently.
4. **Stale bytecode.** Delete all `__pycache__` directories:
 - macOS / Linux: `find . -type d -name __pycache__ -exec rm -rf {} +`
 - PowerShell: `Get-ChildItem -Recurse -Directory -Filter __pycache__ | Remove-Item -Recurse -Force`
5. **The code path isn't actually being executed.** Set a breakpoint earlier in the request lifecycle (the view) and walk down from there.

"Migrations conflict" after a rebase

Two branches added migrations with the same parent. Generate a merge migration:

```
APP_MODE=development python manage.py makemigrations --merge
APP_MODE=development python manage.py migrate
```

Commit the resulting `*_merge_.py` file. Never `git rm` a migration that has already been applied somewhere - write a new migration that undoes its effect instead.

Firebase storage / `NoSuchKey` errors on local upload tests

`STORAGE_BUCKET` in your `.env` is pointing at a bucket your gcloud account can't read or that doesn't exist. Either ask for permission on the dev bucket, or leave `STORAGE_BUCKET=` empty and avoid the file-upload code paths.

WeasyPrint installation issues

WeasyPrint (used for PDF report generation) requires system-level libraries - see §1.7. If pip install still fails after installing the GTK / pango libraries, on Windows the most reliable path is to develop inside WSL2.

`runserver` works, but the admin pages look unstyled

You skipped §2.5. Run `npm install && npm run build` from `djangoexact/`.

8. Day-2 Operations

8.1 Tests

Run the test suite from the Django root:

```
# from: exact-django-webapp/djangoexact/  
APP_MODE=test pytest
```

Useful flags:

- `pytest path/to/test_file.py::TestClass::test_name` runs a single test.
- `pytest -k "annual_cropland"` runs tests matching a keyword.
- `pytest --pdb` drops into the debugger on the first failure.

In VS Code, the **Testing** sidebar discovers `pytest` tests automatically once the interpreter from §2.4 is selected.

8.2 Branching & Pull Requests

- The default branch is `develop`. `main` is protected.
- Branch naming: `feature/<short-name>`, `fix/<short-name>`, `chore/<short-name>`.
- Pull requests target `develop`. The PR description should call out scope, test coverage, and migration notes.
- Commit messages follow Conventional Commits (`fix:`, `feat:`, `chore:`, `docs:`).

8.3 Fixtures & Reference Data

Reference data (IPCC tables, countries, regions, GHG factors) is loaded from JSON fixtures, not migrations:

```
# from: exact-django-webapp/djangoexact/  
APP_MODE=development python manage.py load_reference_data --app=all
```

For the dump/load workflow, PK-stability guardrails, and how to add a new reference model, see [djangoexact/docs/guides/fixtures-guide.md](#).